

Flutter Games & Shaders

Renan Araujo



Renan Araujo

Flutter and Dart **GDE**

OSS dev **BlueFire**

Senior Engineer **Superlist**



Agenda

- Crash Course on Flutter
- Gamedev overview and introduction
- Crash course on Fragment Shaders
- A Look at Flutter Post-processing and a Demo

Deep Diving Talks



“Post Processing API”

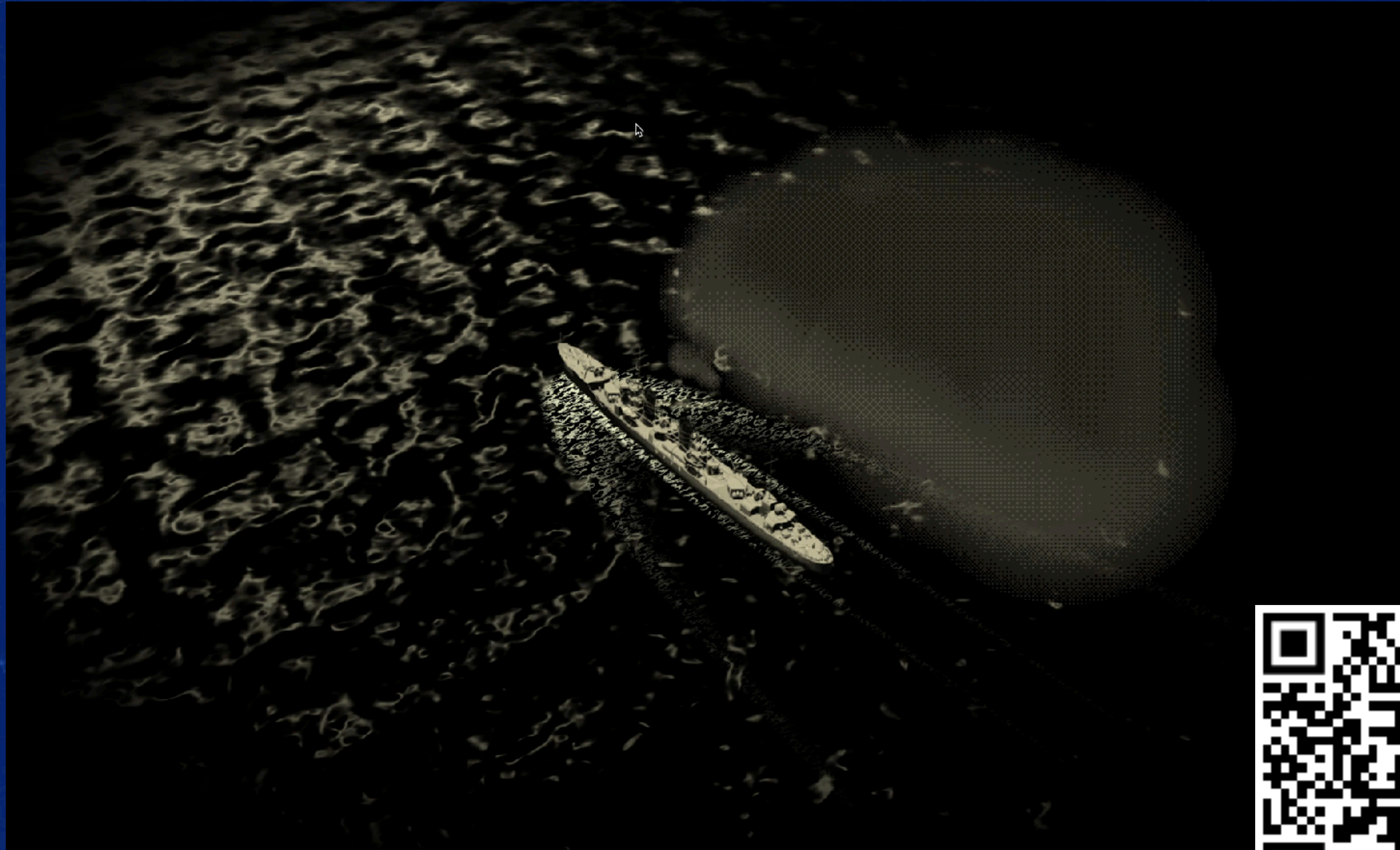
Flutter & Friends • Stockholm

&



**“Flutter, Flame & Fragment
shaders”**

FlutterCon Europe • Berlin



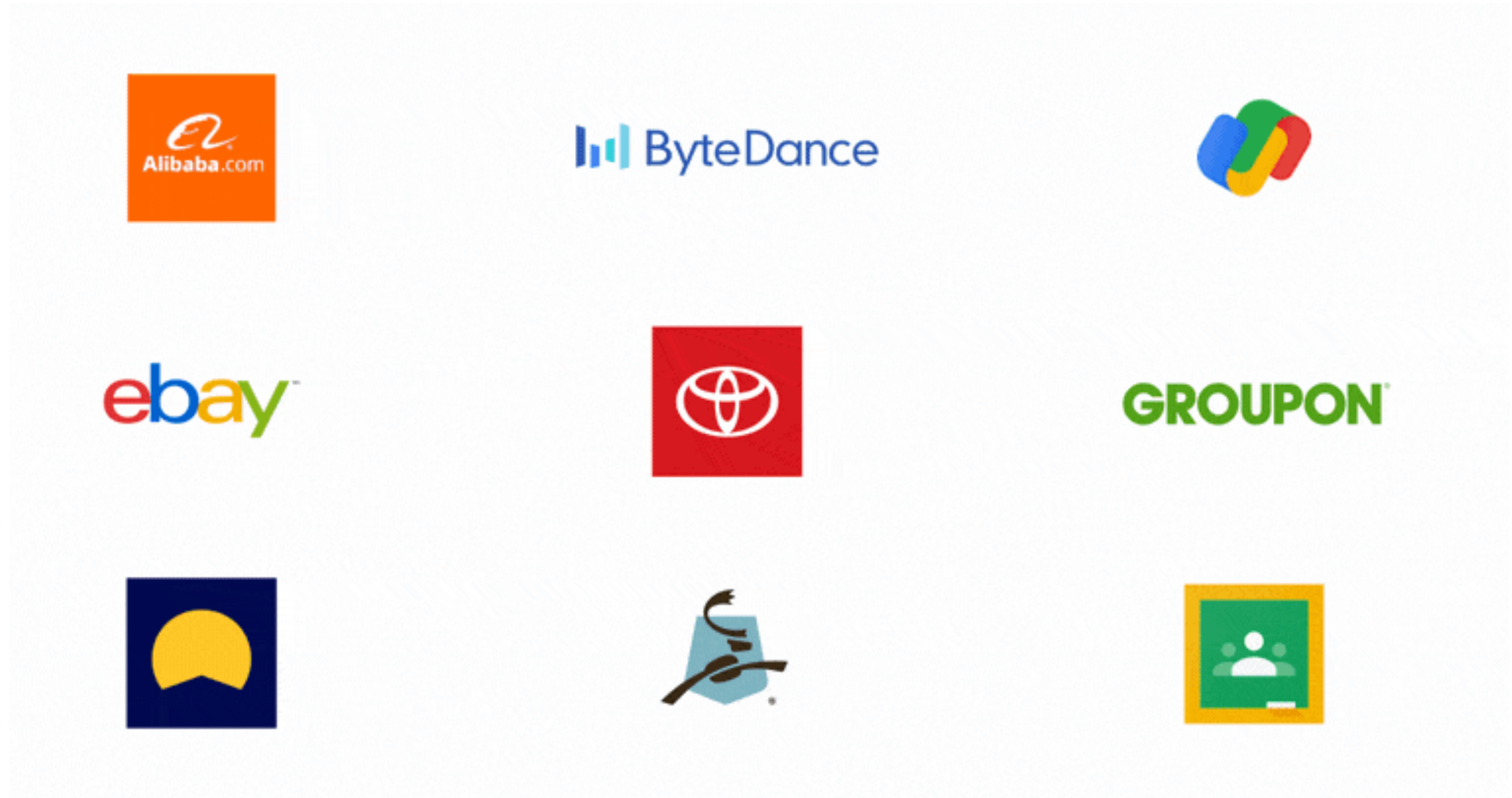
Flutter Games & Shaders

Chapter 1

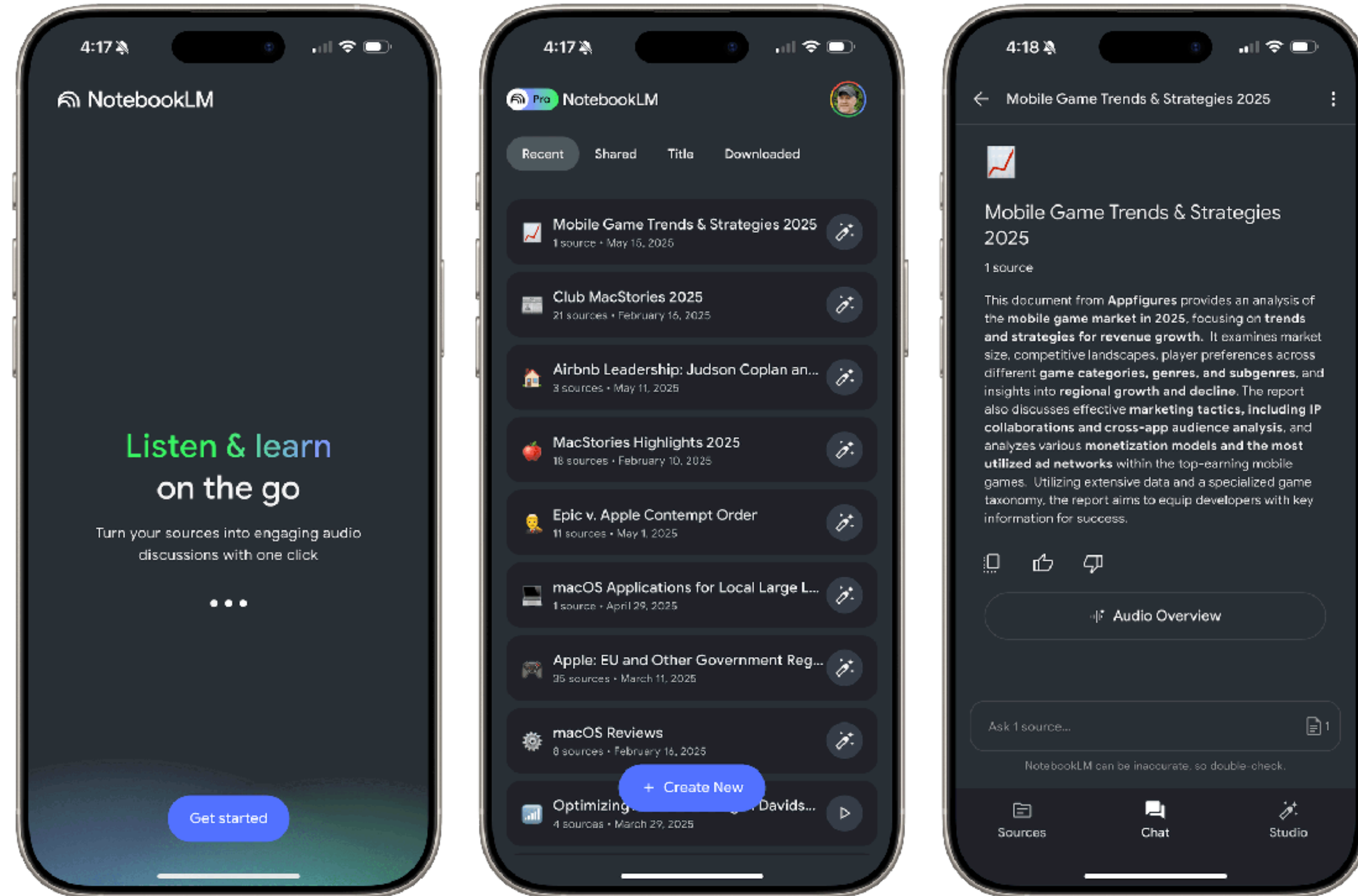
Flutter



- Open Source & Cross-Platform SDK
- Natively compiled applications
- Write once, deploy everywhere



- Very popular on Mobile
- Use cases on Desktop, web, TVs, WearOS
Embedded



How Flutter looks like



Example

Technical details

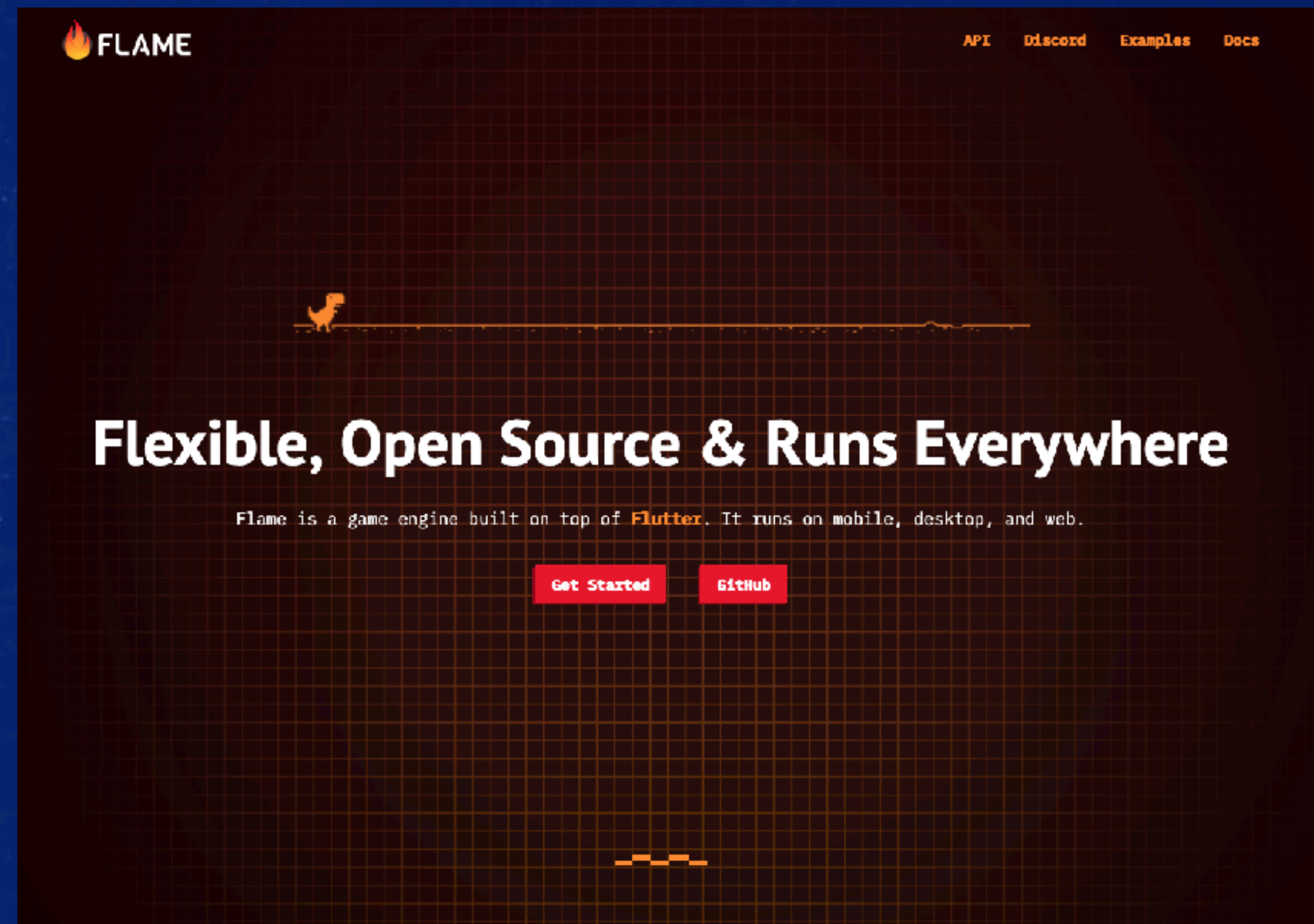
- Written on Dart
- Compiles to fast, native machine code
- Everything is a widget (layout, text, buttons)
- Uses a reactive API

Flutter Games & Shaders

Chapter 2

Games

Flame: Flutter's Game engine



flame-engine.org

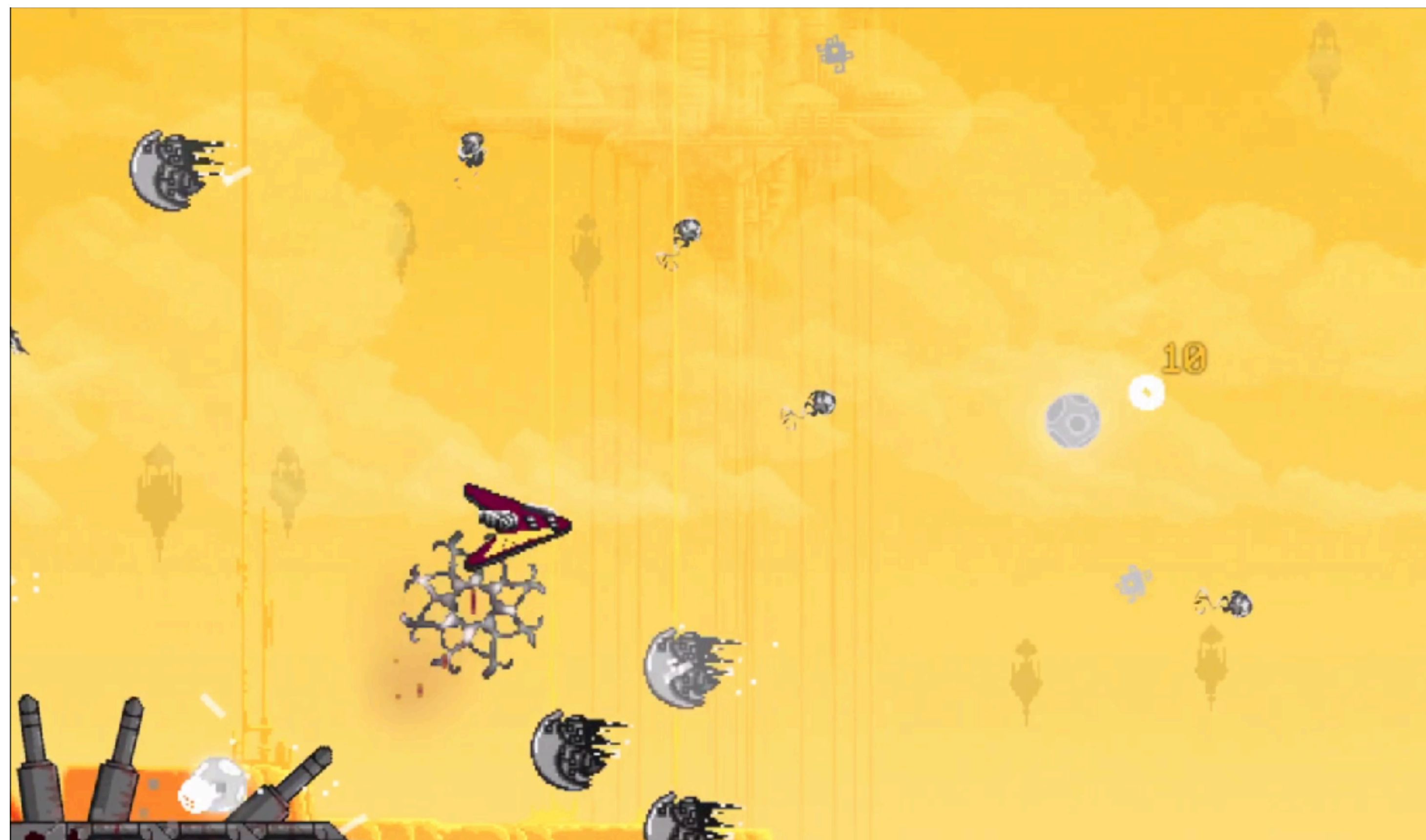
NAN

Flame engine

- Games published for:
 - Mobile
 - Desktop
 - WearOS
 - TVs



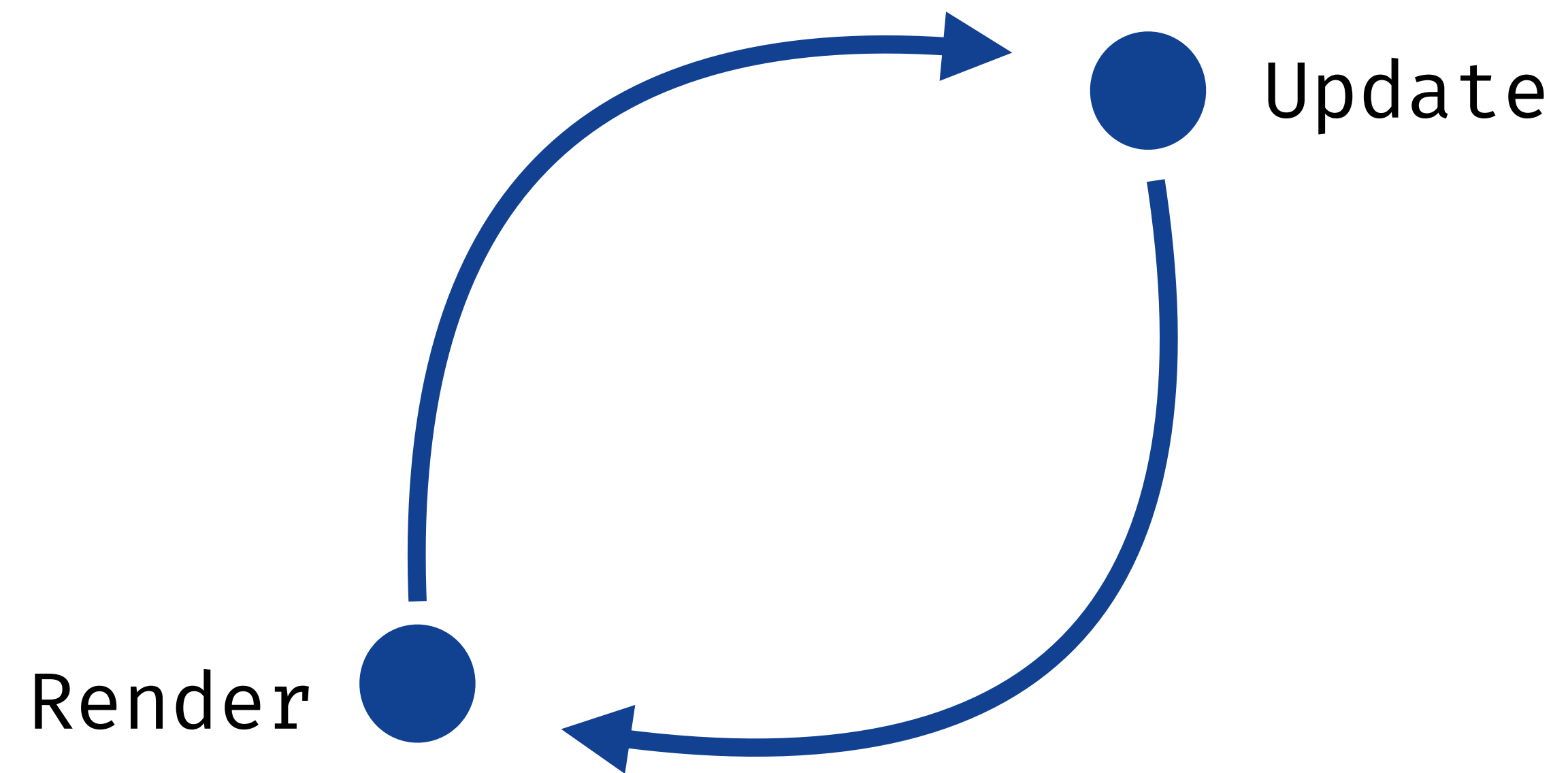
Reactive UIs **VS** **Actively** **updated UIs**



Game: Giga Bull

Game loop

- Like a `while(true)`




```
class MyGame extends FlameGame {
```

```
// ...
```

```
GameWidget(game: MyGame())
```

```
class MyComponent extends PositionComponent {  
    MyComponent() {  
        add(AnotherComponent());  
    }  
  
    @override  
    void update(double dt) {  
        // Update component state  
    }  
  
    @override  
    void render(Canvas canvas) {  
        // Render component  
    }  
}
```


Live code example

Flutter Games & Shaders

Chapter 3

Shaders

```
#include <flutter/runtime_effect.glsl>

uniform vec2 uSize;

out vec4 fragColor;

void main() {
    vec2 pos = FlutterFragCoord().xy;
    vec2 uv = pos / uSize;

    float red = uv.x;
    float green = uv.y;
    float blue = 1.0 - red;

    fragColor = vec4(red, green, blue, 1.0);
}
```

```
#include <flutter/runtime_effect.glsl>
```

```
uniform vec2 uSize;
```

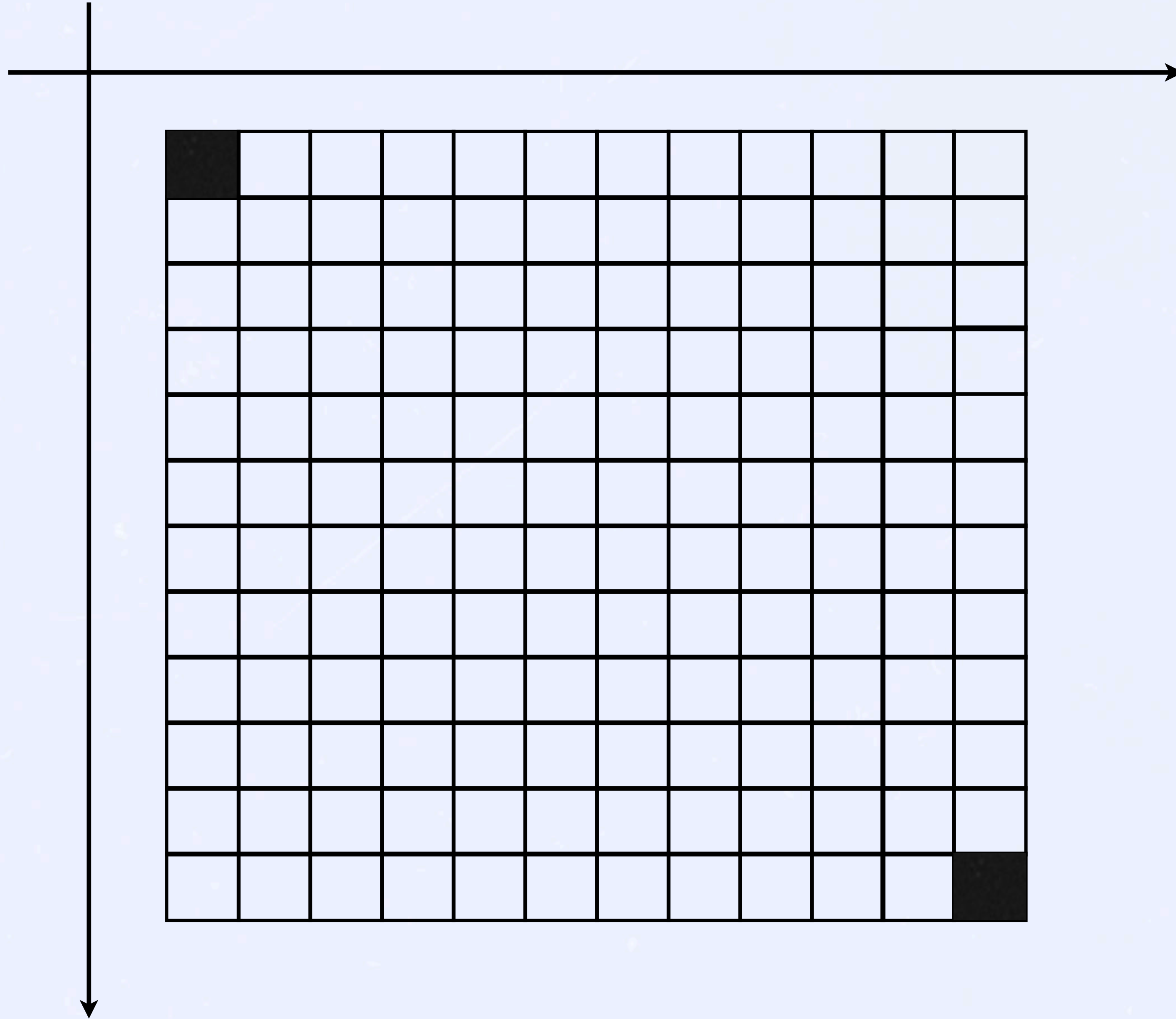
```
out vec4 fragColor;
```

```
void main() {  
    vec2 pos = FlutterFragCoord().xy;  
    vec2 uv = pos / uSize;
```

```
    float red = uv.x;  
    float green = uv.y;  
    float blue = 1.0 - red;
```

```
    fragColor = vec4(red, green, blue, 1.0);  
}
```

What color should a pixel have?



```
vec2 uv = vec2(0.0, 0.0);
```

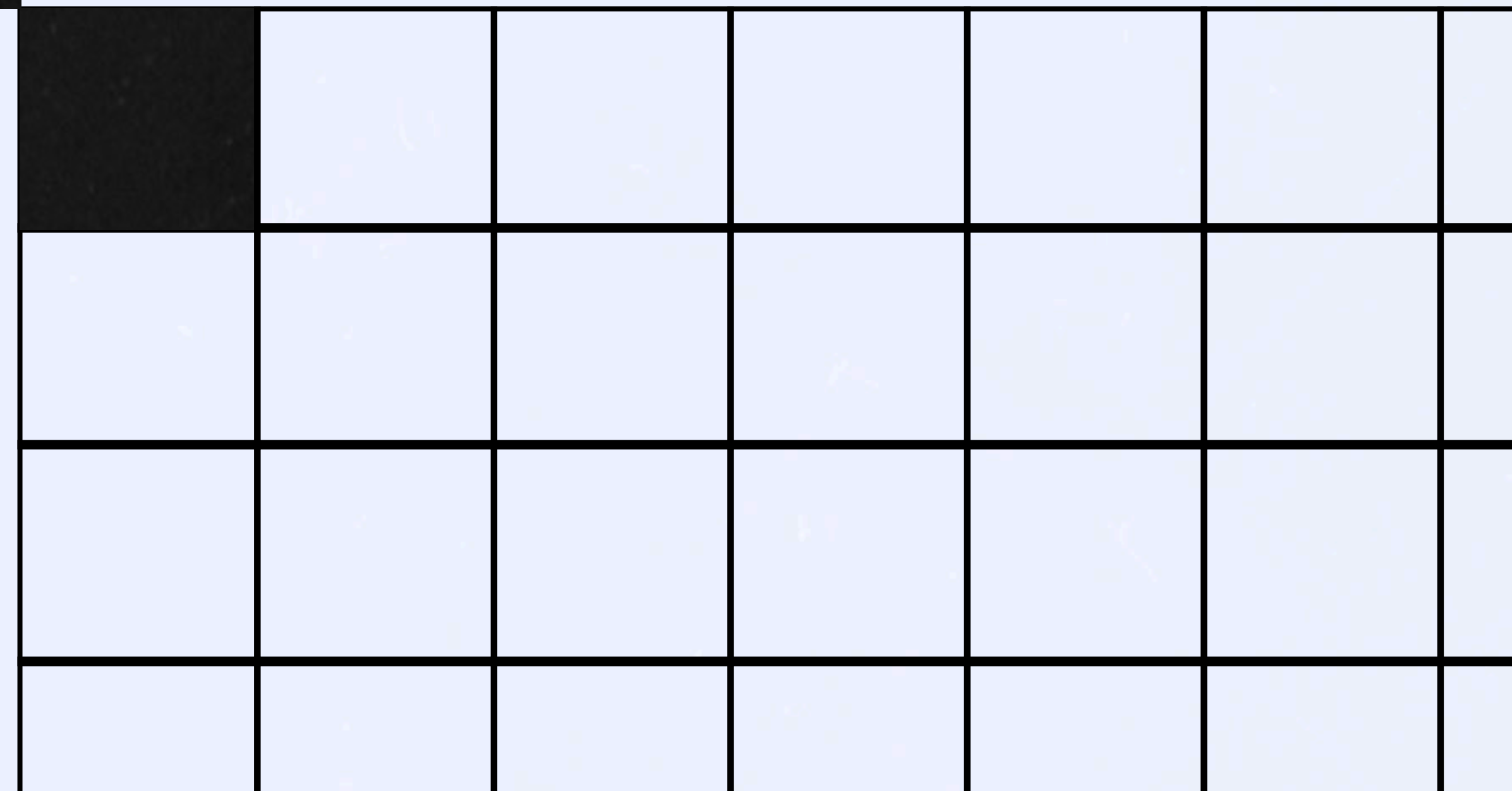
```
float red = uv.x;
```

```
float green = uv.y;
```

```
float blue = 1.0 - red;
```

```
fragColor = vec4(red, green, blue, 1.0);
```

NAN



```
vec2 uv = vec2(0.0, 0.0);
```

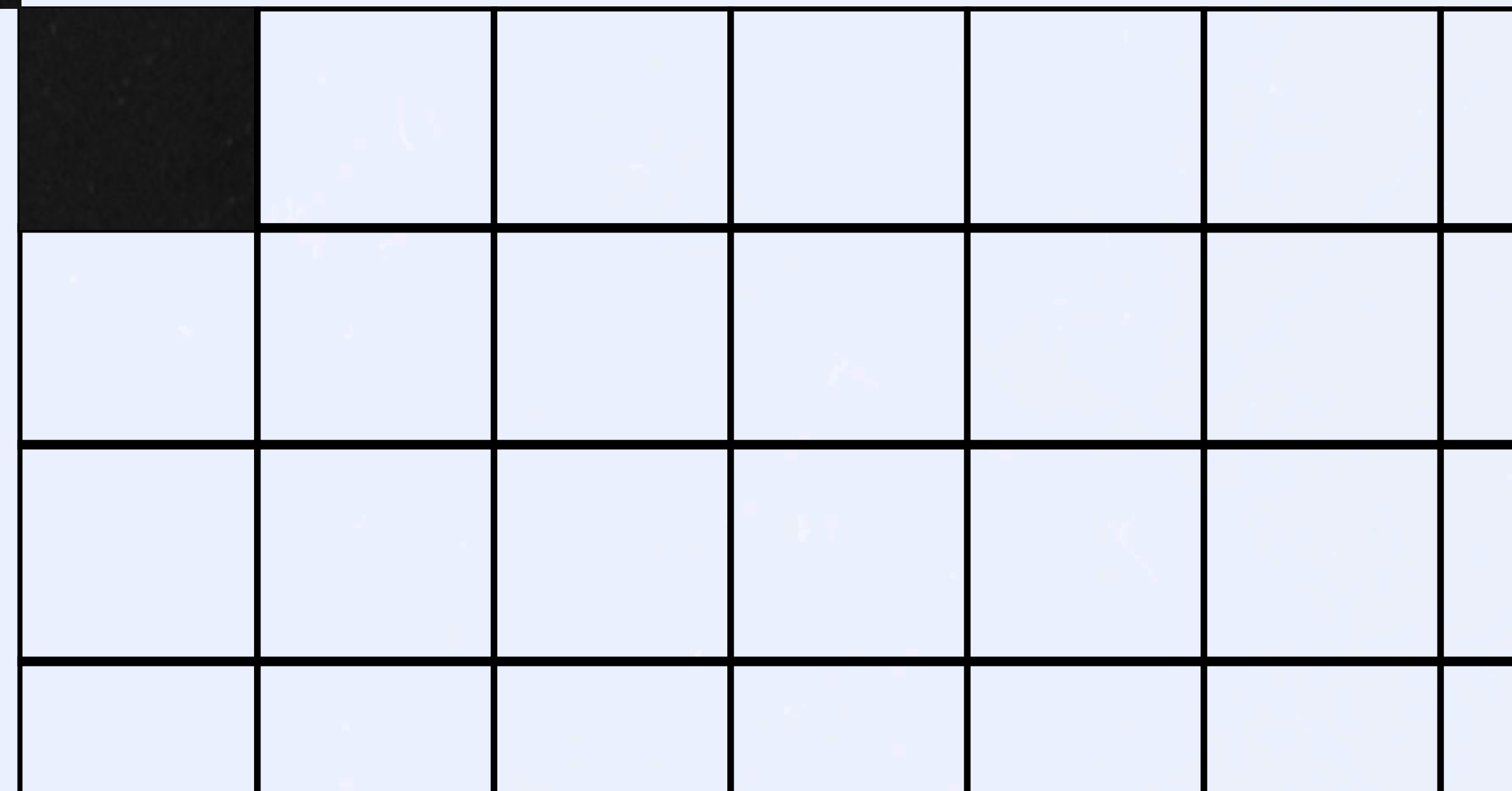
```
float red = 0.0;
```

```
float green = 0.0;
```

```
float blue = 1.0 - 0.0;
```

```
fragColor = vec4(red, green, blue, 1.0);
```

NAN




```
vec2 uv = vec2(0.0, 0.0);
```

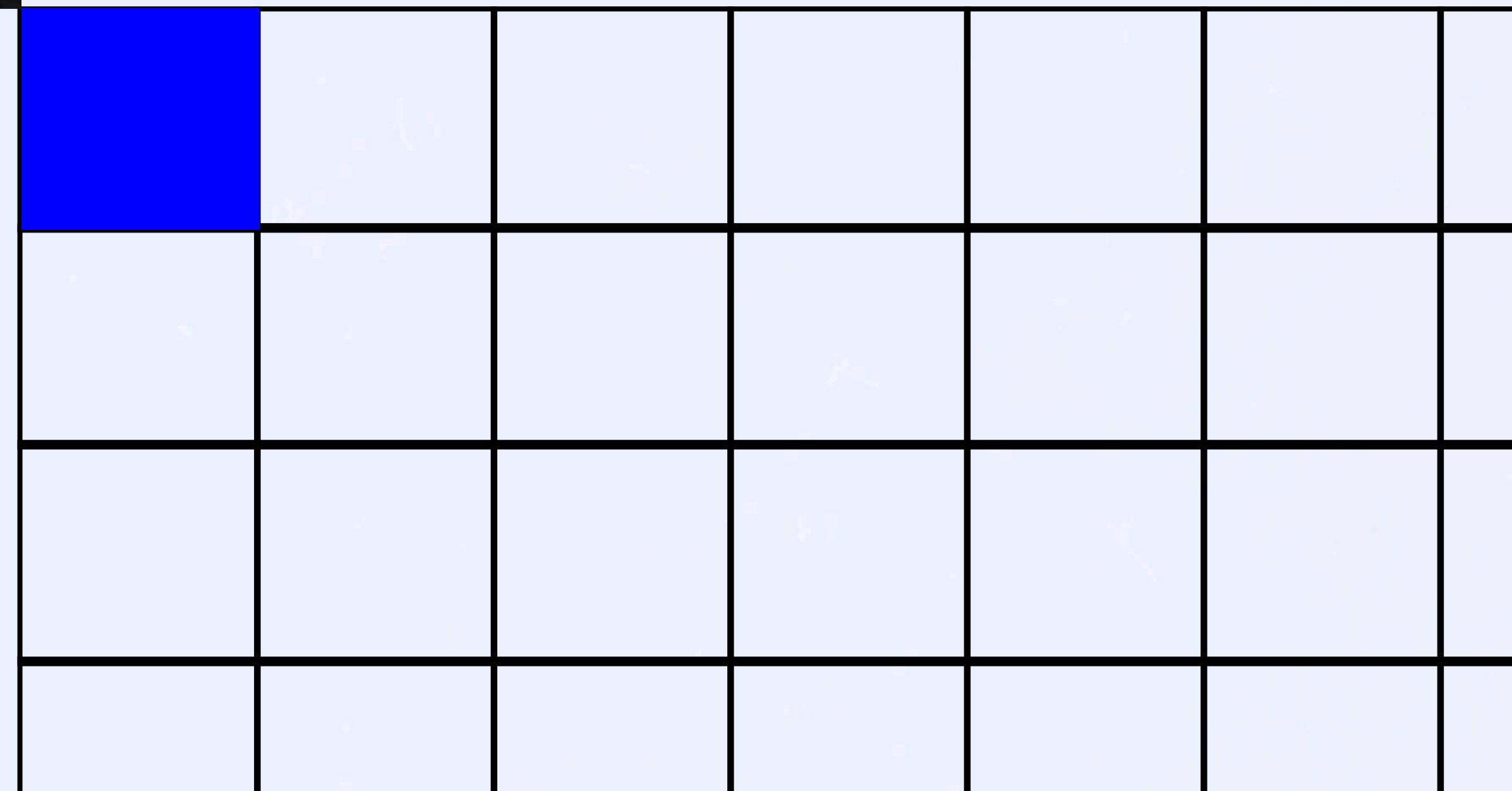
```
float red = 0.0;
```

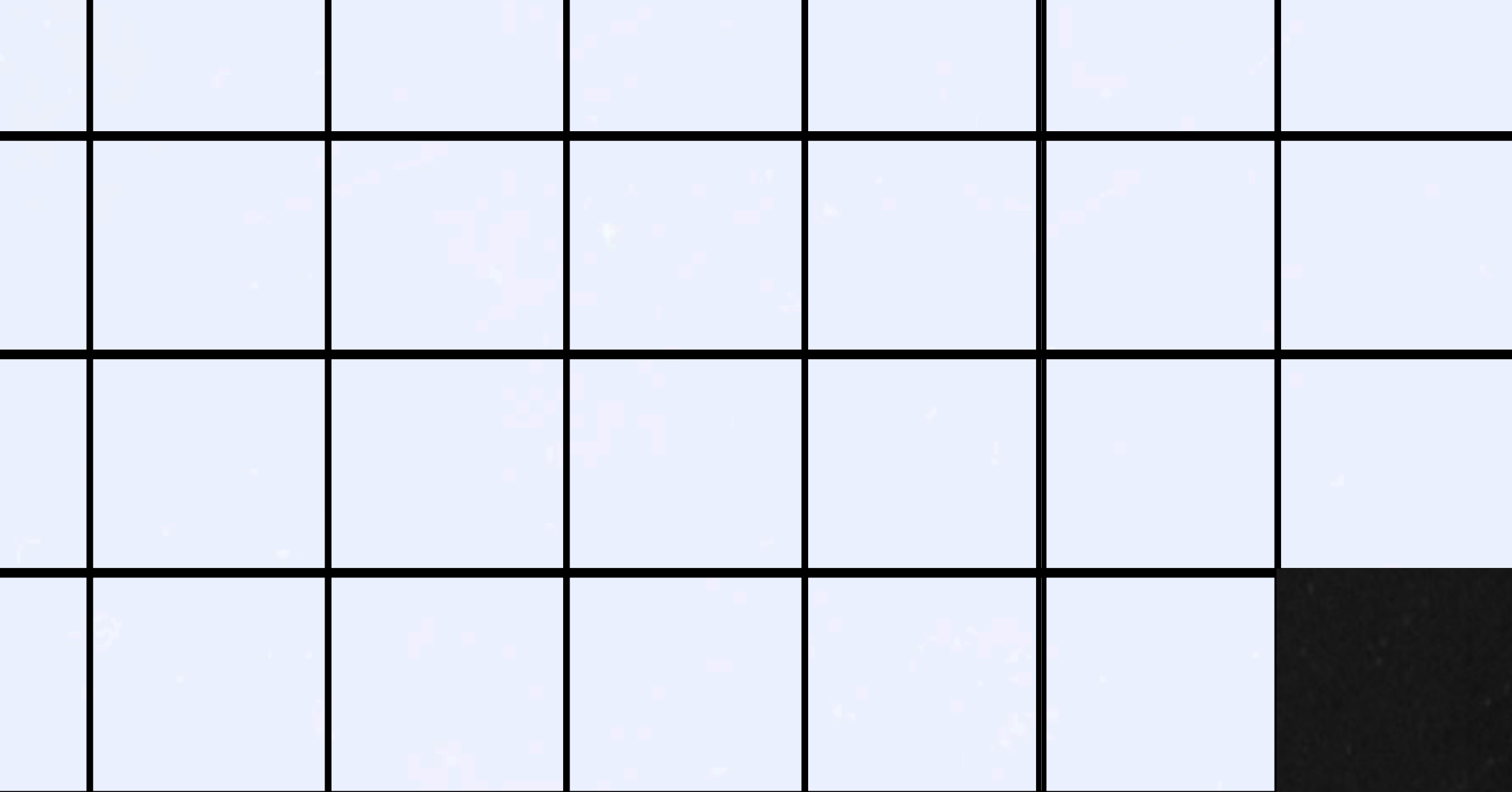
```
float green = 0.0;
```

```
float blue = 1.0 - 0.0;
```

```
fragColor = vec4(red, green, blue, 1.0);
```

NAN



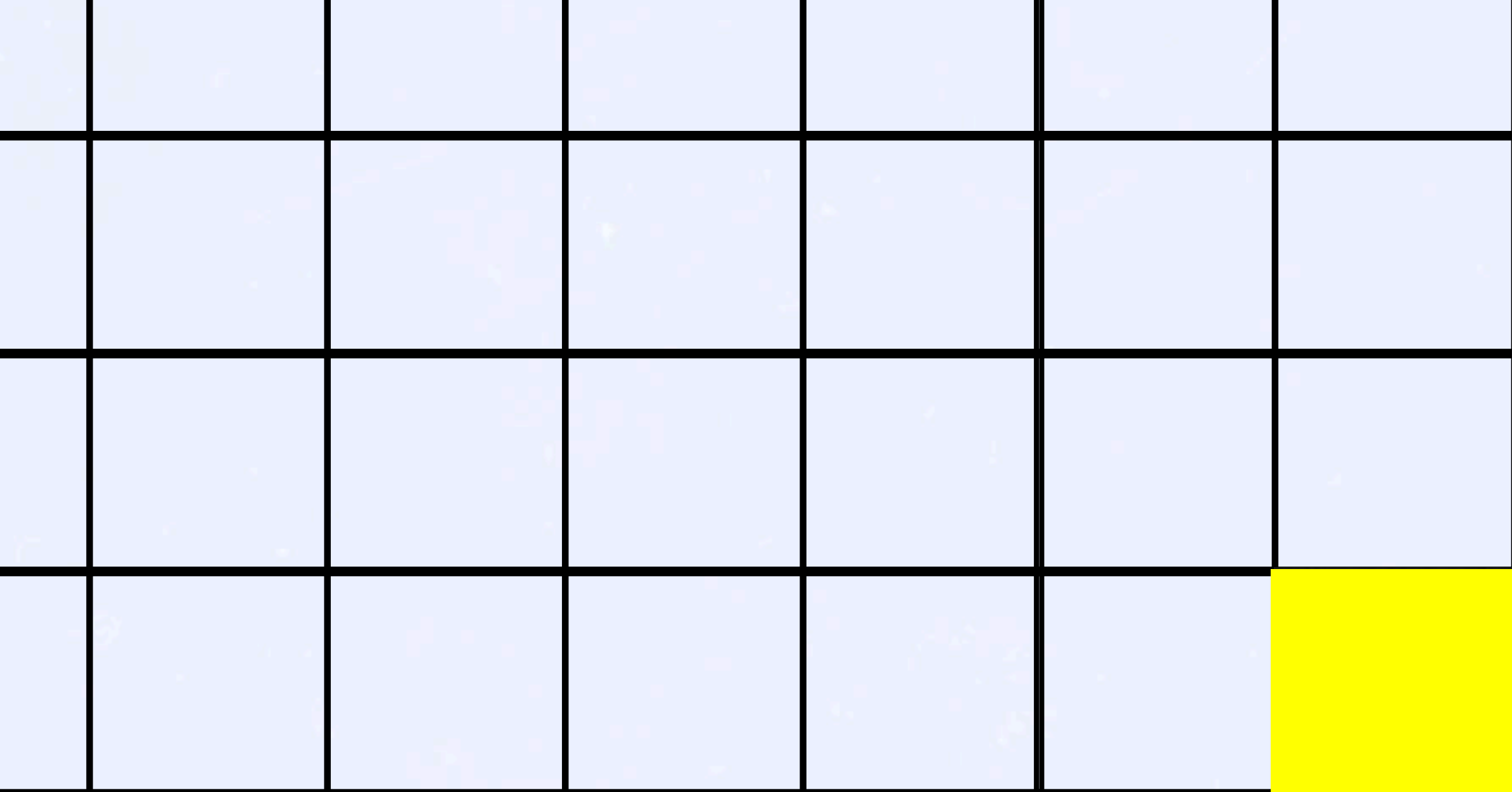


```
vec2 uv = vec2(1.0, 1.0);
```

```
float red = uv.x;  
float green = uv.y;  
float blue = 1.0 - red;
```

```
fragColor = vec4(red, green, blue, 1.0);
```

NAN



```
vec2 uv = vec2(1.0, 1.0);
```

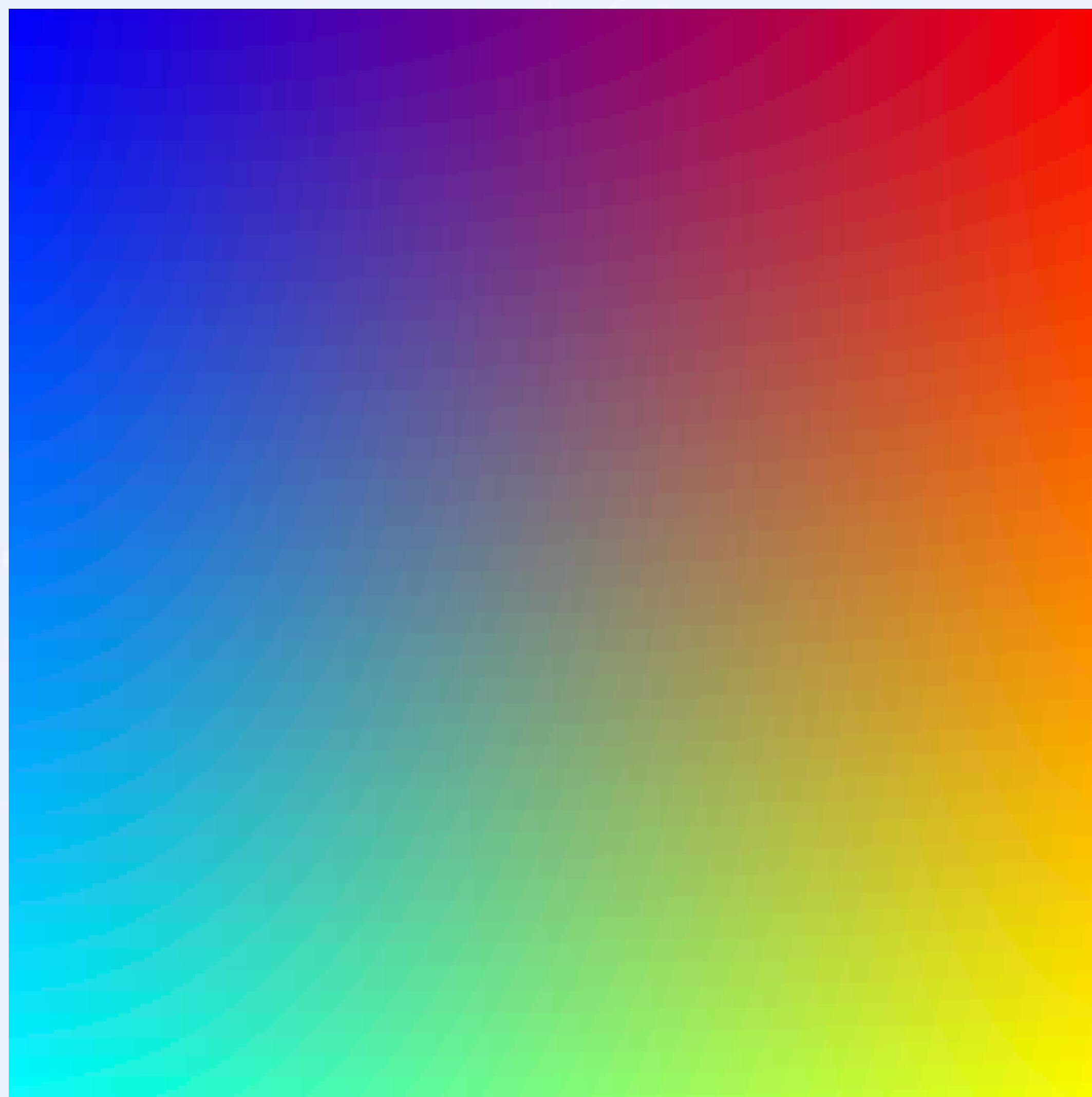
```
float red = 1.0;
```

```
float green = 1.0;
```

```
float blue = 1.0 - 1.0;
```

```
fragColor = vec4(red, green, blue, 1.0);
```

NAN



NAN

```
uniform float uScale;  
uniform vec2 uSize;  
uniform vec4 uColor;  
uniform mat4 uTransform;  
uniform sampler2D uTexture;
```

Inputs

- A series of numeric types
(float, vectors, matrices, arrays...)
- Textures

Possible inputs

- Scroll position
- Colors
- Gesture coordintes
- Sizes
- Animated values

Chapter 4

Flutter Games & Shaders

Ways to render Shaders on Flutter

1. Just paint it

NAN

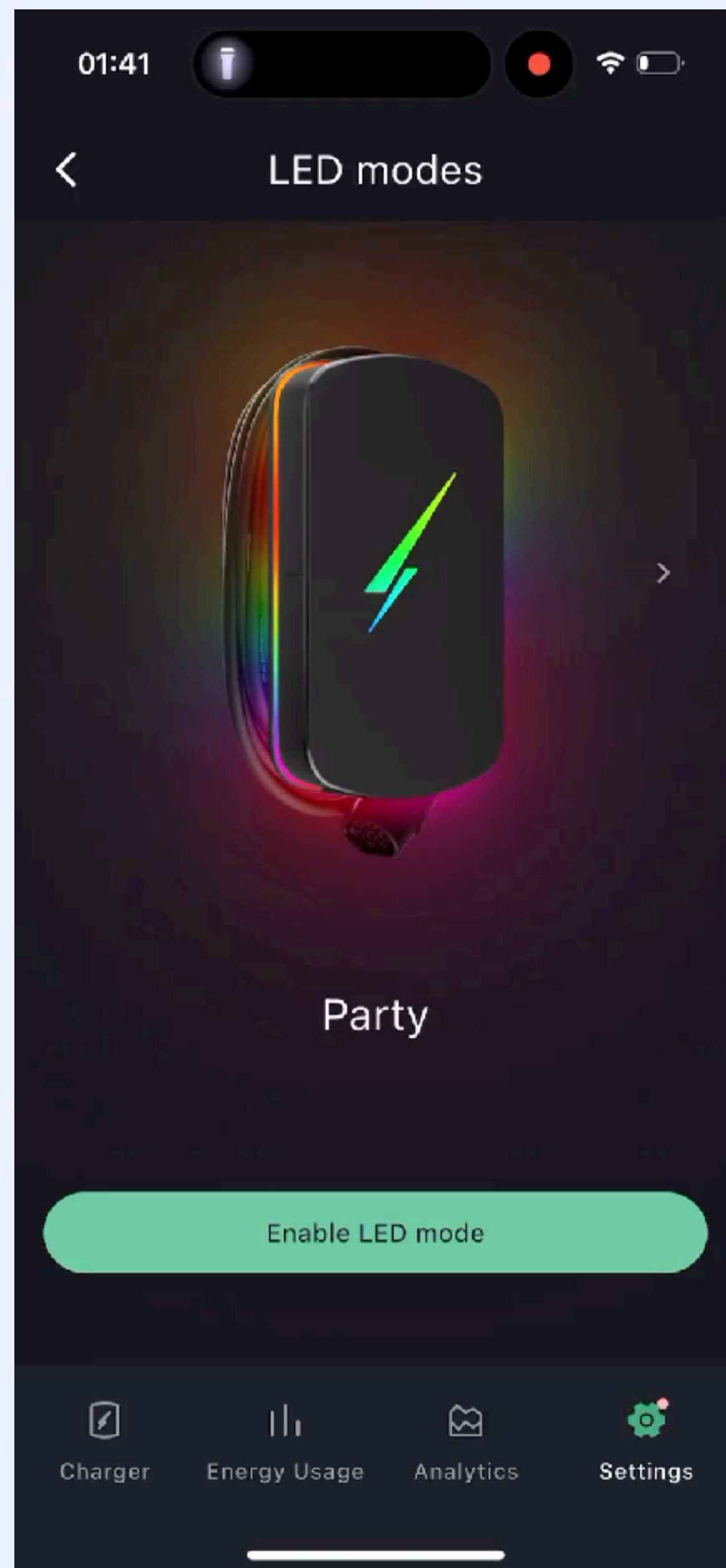

```
final paint = Paint()  
    ..shader = shader;
```

```
shader.setFloatUniforms((s) {  
    s.setSize(size);  
});
```

```
canvas.drawRect(Offset.zero & size, paint);
```

Canvas based APIs

CustomPainter, Render Objects



Hypervolt

Render light effects

hypervolt.co.uk

NAN

renan.gg

2. Use UI as Input to the shader

12:39

Show Superellipse

22:26

22:26

22:26

3. Post process on Flame



```
void main() {  
    runApp(GameWidget(game: MyGame()));  
}
```

```
class MyGame extends FlameGame {  
    @override  
    Future<void> onLoad() async {  
        world.add(  
            EmberPlayer(),  
        );  
    }  
}
```



```
#version 460 core

precision highp float;

#include <flutter/runtime_effect.glsl>

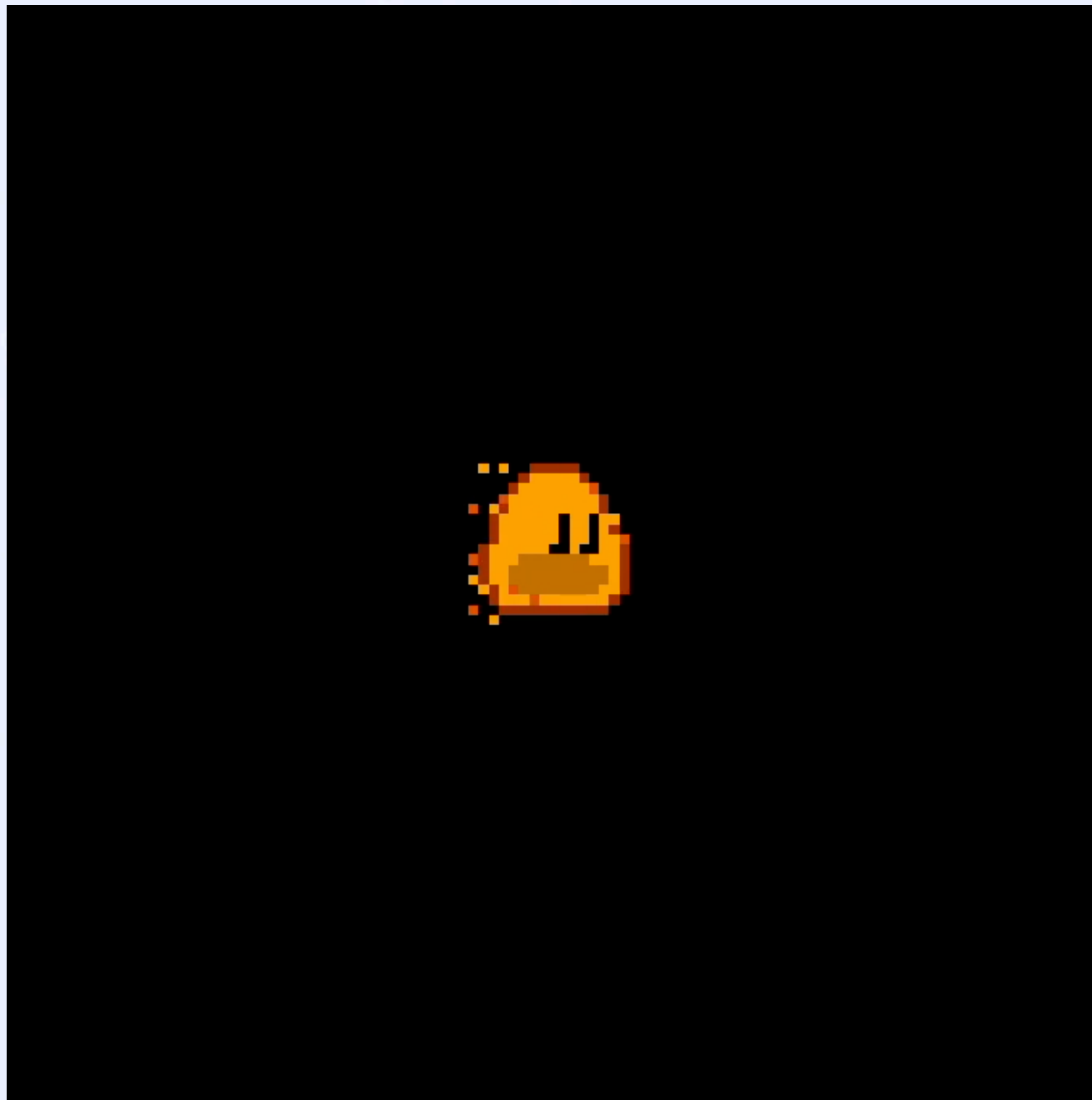
uniform vec2 uSize;
uniform vec2 uPixels;
uniform sampler2D uTexture;

out vec4 fragColor;

void main() {
    vec2 uv = FlutterFragCoord().xy / uSize;
    vec2 puv = round(uv * uPixels) / uPixels;
    fragColor = texture(uTexture, puv);
}
```




```
class MyGame extends FlameGame {  
  @override  
  Future<void> onLoad() async {  
    world.add(  
      PostProcessComponent(  
        postProcess: PixelationPostProcess(),  
        children: [EmberPlayer()],  
      ),  
    );  
  }  
}
```

```
import 'package:flame/post_process.dart';
```

```
class MyGame extends FlameGame {  
  @override  
  Future<void> onLoad() async {  
    world.add(  
      PostProcessComponent(  
        postProcess: PixelationPostProcess(),  
        children: [EmberPlayer()],  
      ),  
    );  
  }  
}
```




```
import 'package:flame/post_process.dart';
```

```
class PixelationPostProcess extends PostProcess {  
  ...  
}
```

NAN



NAN

```
class PixelationPostProcess extends PostProcess {
```

```
    @override
```

```
    Future<void> onLoad() async {
```

```
        _fragmentShader = (await ui.FragmentProgram.fromAsset(
            'shaders/my_shader.frag',
        )).fragmentShader();
    }
```

```
    late final ui.FragmentShader _fragmentShader;
```

```
    late final myPaint = Paint()..shader = _fragmentShader;
```

```
    @override
```

```
    void postProcess(Vector2 size, Canvas canvas) {
```

```
    }
```




```
@override
void postProcess(Vector2 size, Canvas canvas) {

    final ui.Image preRenderedSubtree = rasterizeSubtree();

    _fragmentShader.setFloatUniforms((value) {
        value
            ..setVector(size)
            ..setVector(Vector2(4, 4));
    });

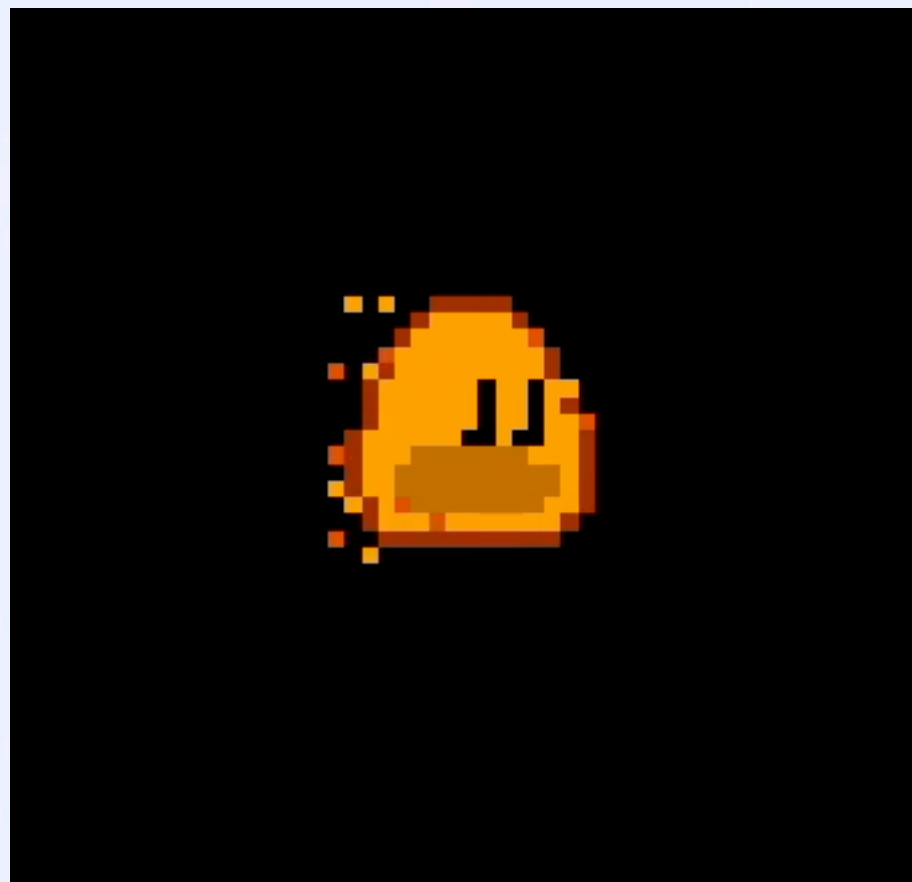
    _fragmentShader.setImageSampler(0, preRenderedSubtree);

    canvas
        ..save()
        ..drawRect(ui.Offset.zero & size.toSize(), myPaint)
        ..restore();

}
```



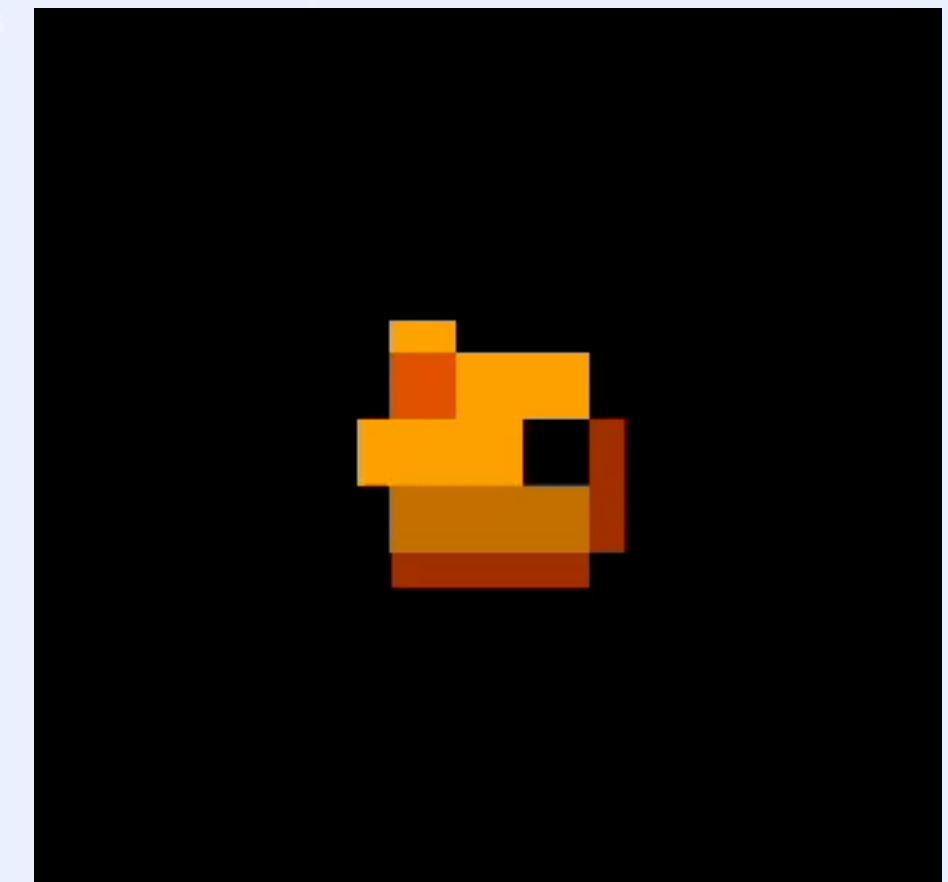
Component renders



Post-process
sends image to
shader



Shader renders



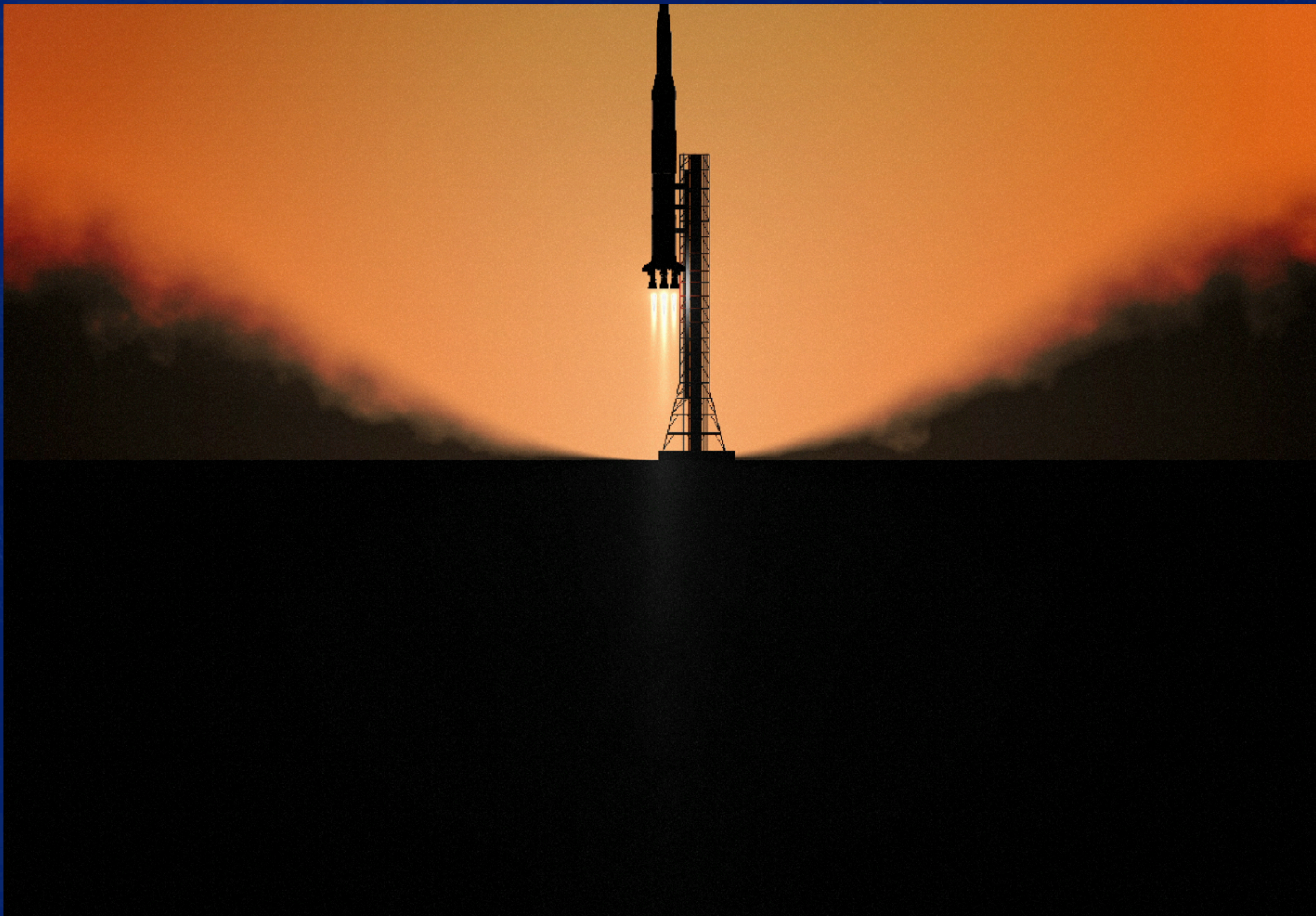
Official shader example



examples.flame-engine.org

NAN





NAN

NAN

renan.gg